

UNIT-III

Advanced Software Engineering Concepts

25MCA203: SOFTWARE ENGINEERING AND TESTING

Software Engineering Paradigms

Software engineering paradigms define how software is designed, developed, and organized

They help in managing complexity and structuring software systems

Paradigms guide work from requirements to maintenance

Software Engineering Paradigms (Conceptual Approaches)

1. Procedural Paradigm

- Software is organized as a sequence of procedures or functions
- Emphasis on step-by-step execution
- Example: Payroll processing system using functions

2. Object-Oriented Paradigm

- Software is modeled using objects and classes
- Supports encapsulation, inheritance, and polymorphism
- Example: Banking system with Account and Customer objects

3. Functional Paradigm

- Focuses on functions and immutability
- Avoids shared data and side effects
- Example: Data transformation using pure functions

Software Engineering Paradigms (Conceptual Approaches)

4. Data-Centered (Data-Driven) Paradigm

- Software behavior revolves around data and databases
- Applications are built around data storage and retrieval
- Example: Inventory management system driven by database operations

5. Agile Paradigm

- Emphasizes iterative development and customer feedback
- Flexible to changing requirements
- Example: Mobile app developed in short sprints

6. DevOps Paradigm

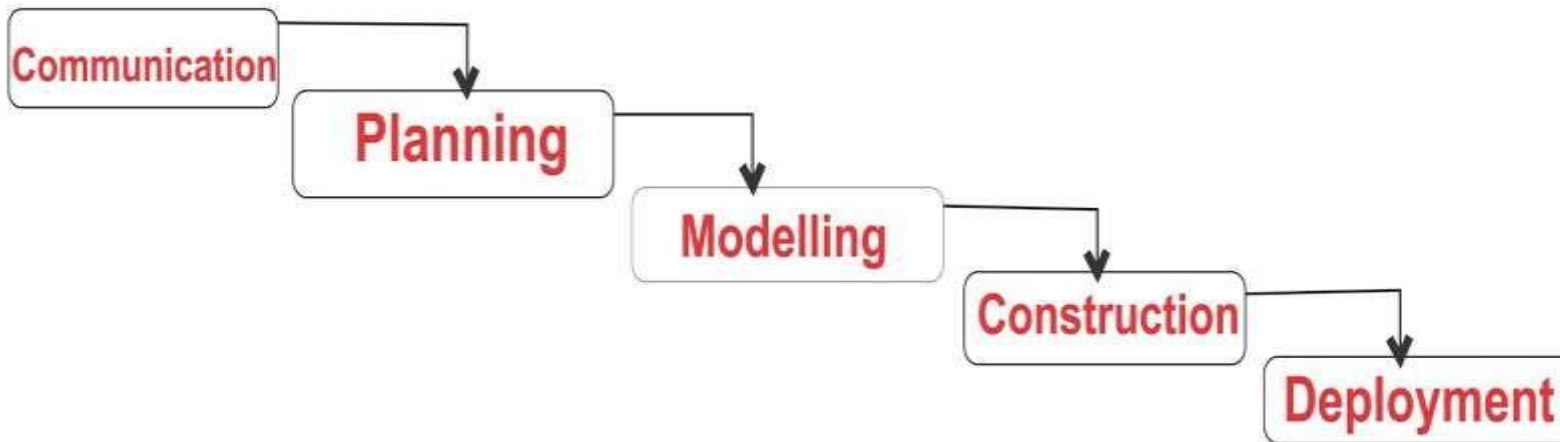
- Integrates development and operations
- Focuses on automation and continuous delivery
- Example: Automated deployment of a web application

Software Development Models (Process Flows/SDLCs)

- Every software engineering organization should describe a unique set of framework activities for the software process it adopts.
 - Waterfall Life Cycle Model.
 - Iterative Waterfall Life Cycle Model.
 - Prototype Model.
 - RAD Model.
 - Spiral Model.
 - Incremental Model.

Waterfall Life Cycle Model.

- It is called classic life cycle or Linear model.
- Requirements are well defined and stable.
- It suggests a systematic, sequential approach to software development.
- It begins with customer specification of requirements and progresses.
 - Planning.
 - Modeling.
 - Construction and
 - Deployment.



WaterFall Model

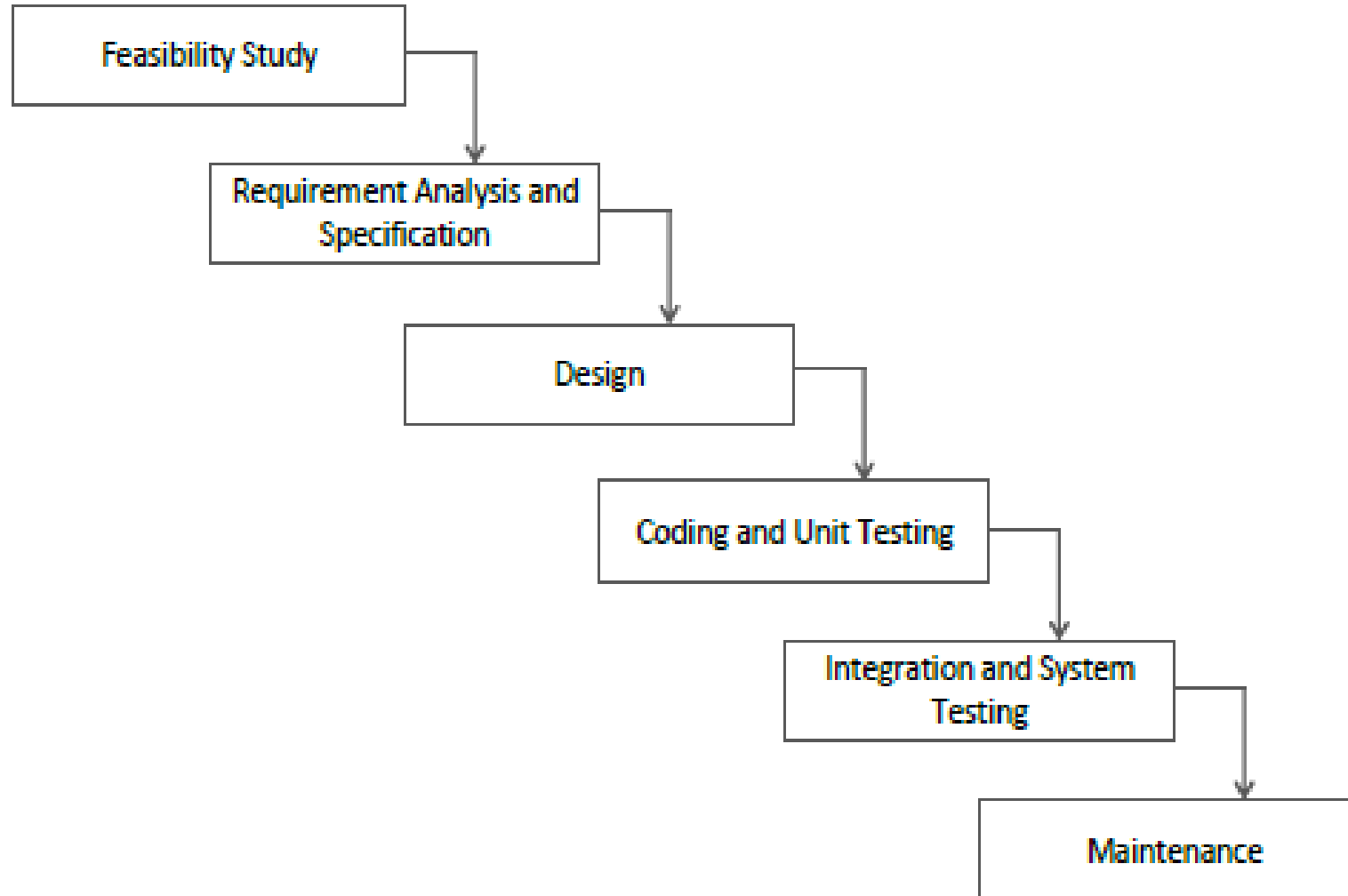
Advantages:

- Easy to understand.
- Each phase has well defined input and output.
- Helps project manager in proper planning of project.
- Provides a templates into which methods of analysis, design, code and support can be placed.

Disadvantages:

- One way street.
- It lack overlapping and interactions among phases.
- Model doesn't support delivery of system in pieces.

Phases of the Classical Waterfall Model:



Feasibility Study:

- It involves analysis of the problem and collection of all relevant information relating to the product.
- The collected data are analysed.
 - Requirements of the Customer.
 - Formulations of the different strategies for solving the problem.
 - Evaluation of different solution strategies.

Requirements Analysis and Specification:

- It is understand the exact requirements of the customer and to document them properly.
 - Requirements gathering and analysis.
 - Requirements specification.

Design:

- The design phase is to transform the requirements specified in the document into a structure that is suitable for implementation in some programming language.
 - Traditional Design Approach.
 - Object-Oriented Design Approach.

Coding and Unit Testing:

- The purpose of the coding and unit testing phase of software development is to translate the software design into source code.

Integration and System Testing:

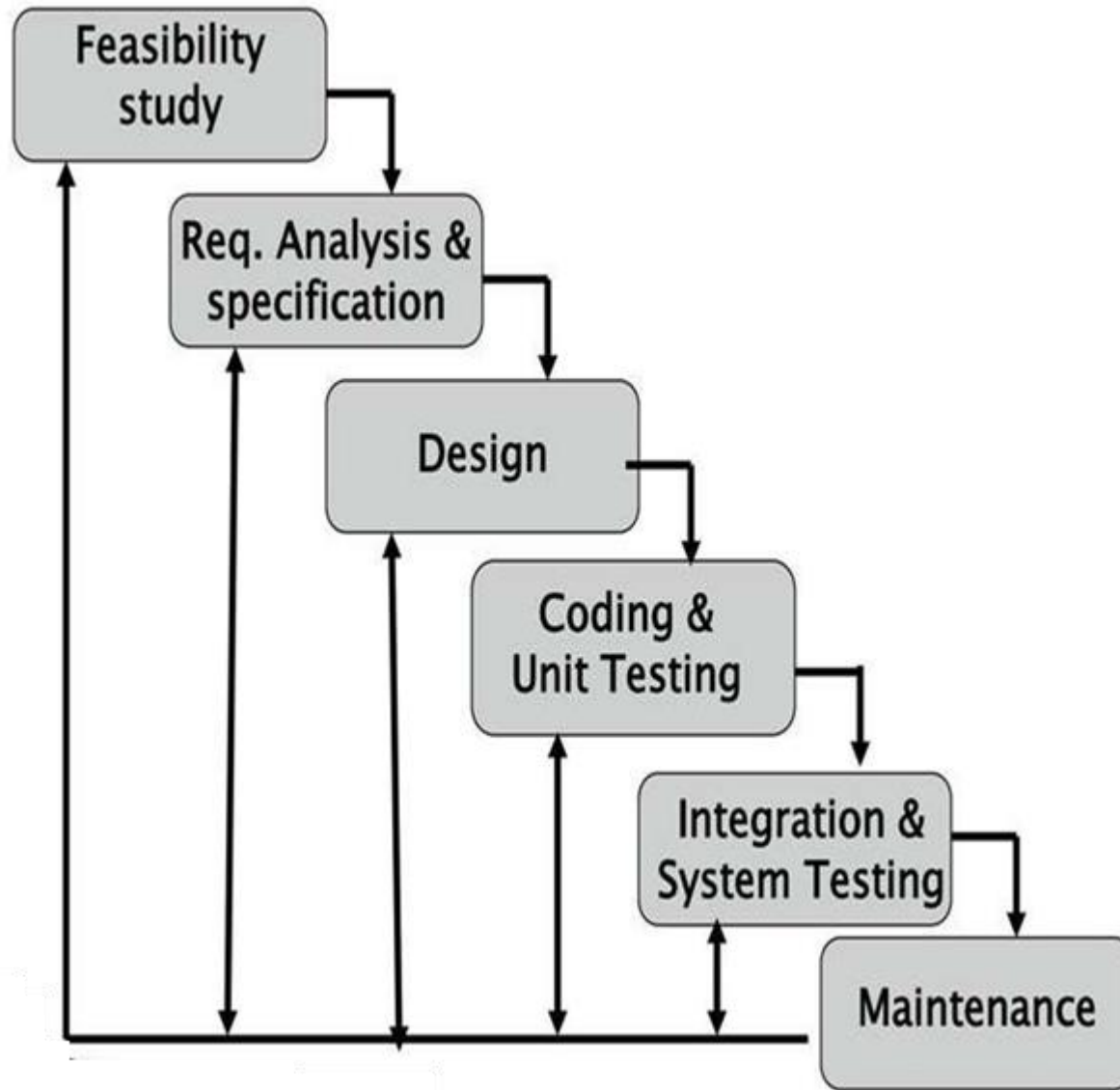
- 'Integration of different modules is coded and unit tested.'
 - α – Testing
 - β – Testing
 - Acceptance Testing.

Maintenance:

- Maintenance of a typical software products requires much more than the effort necessary to develop the product itself.

Iterative Waterfall life cycle model:

- The main changes is done by providing feedback paths from every phase to its preceding phase.



Prototype Model:

A software prototyping model is a development model in which a prototype is built, tested, and reworked until an acceptable prototype is achieved.

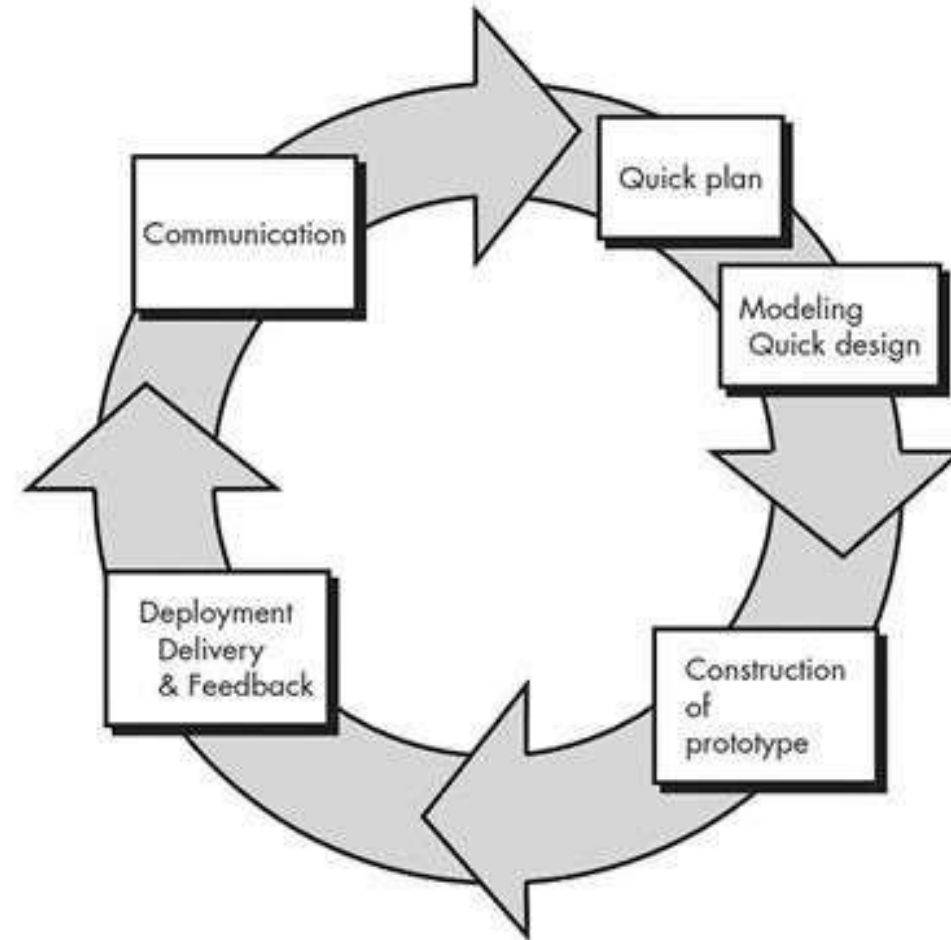


Figure: Prototype Model

Advantages:

- Clarity.
- Risk Identification.
- Good Environment.
- Take less time to complete.

Disadvantages:

- High cost.
- Slow process.
- Too many changes.

RAD Model:

- Rapid Application Development(RAD) is an incremental software model that a short development cycle.
- The RAD model is a “high-speed” of the waterfall model.
- The RAD process enables a development team to create a fully functional system within a very short time period.

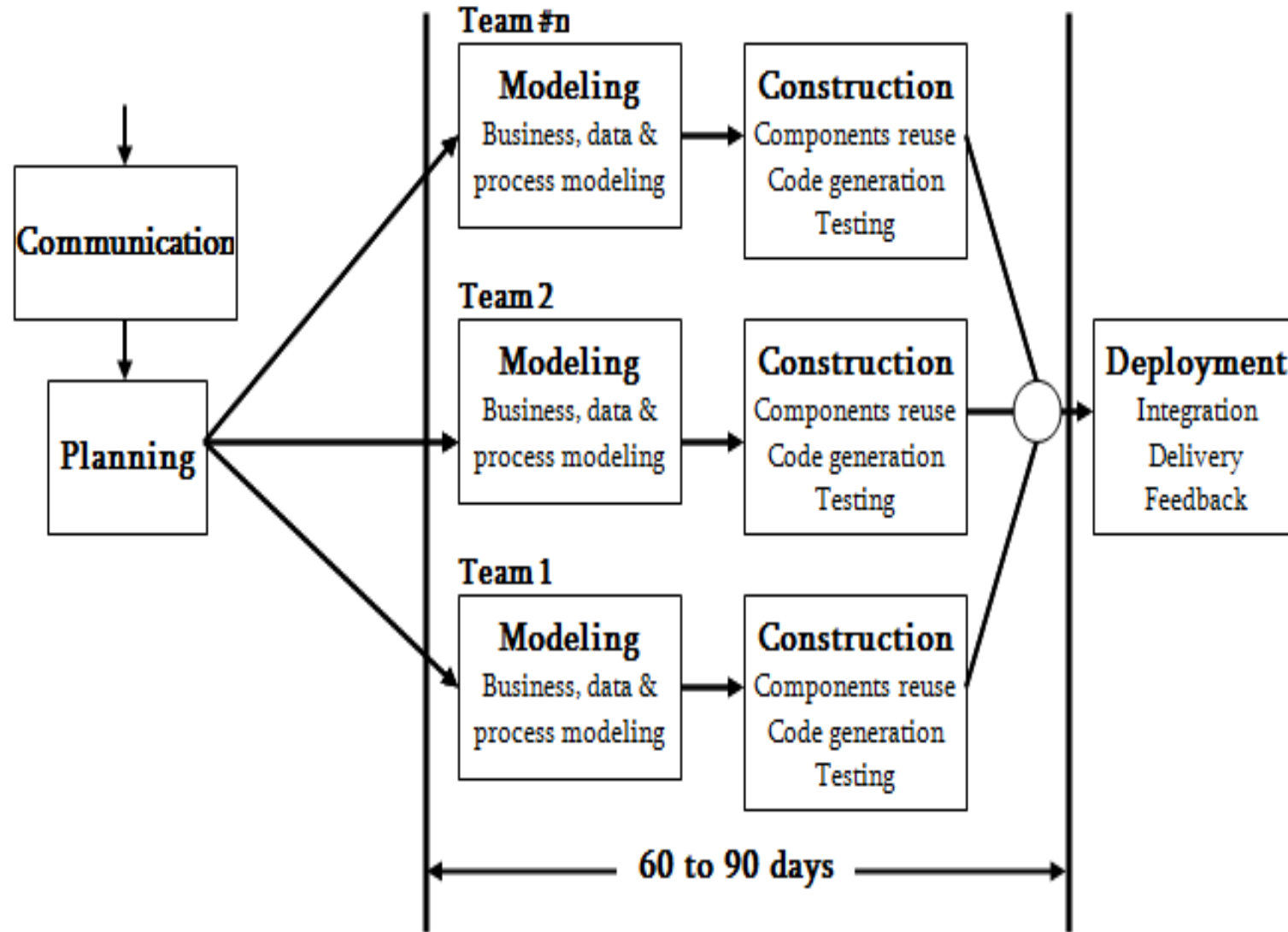


Figure : Flowchart of RAD model

Contents of RAD Packages:

- Graphical user development environment.
- Reusable Components.
- Code generator.
- Programming Language.

Advantages:

- Fast products.
- Efficient Documentation.
- Interaction with user.

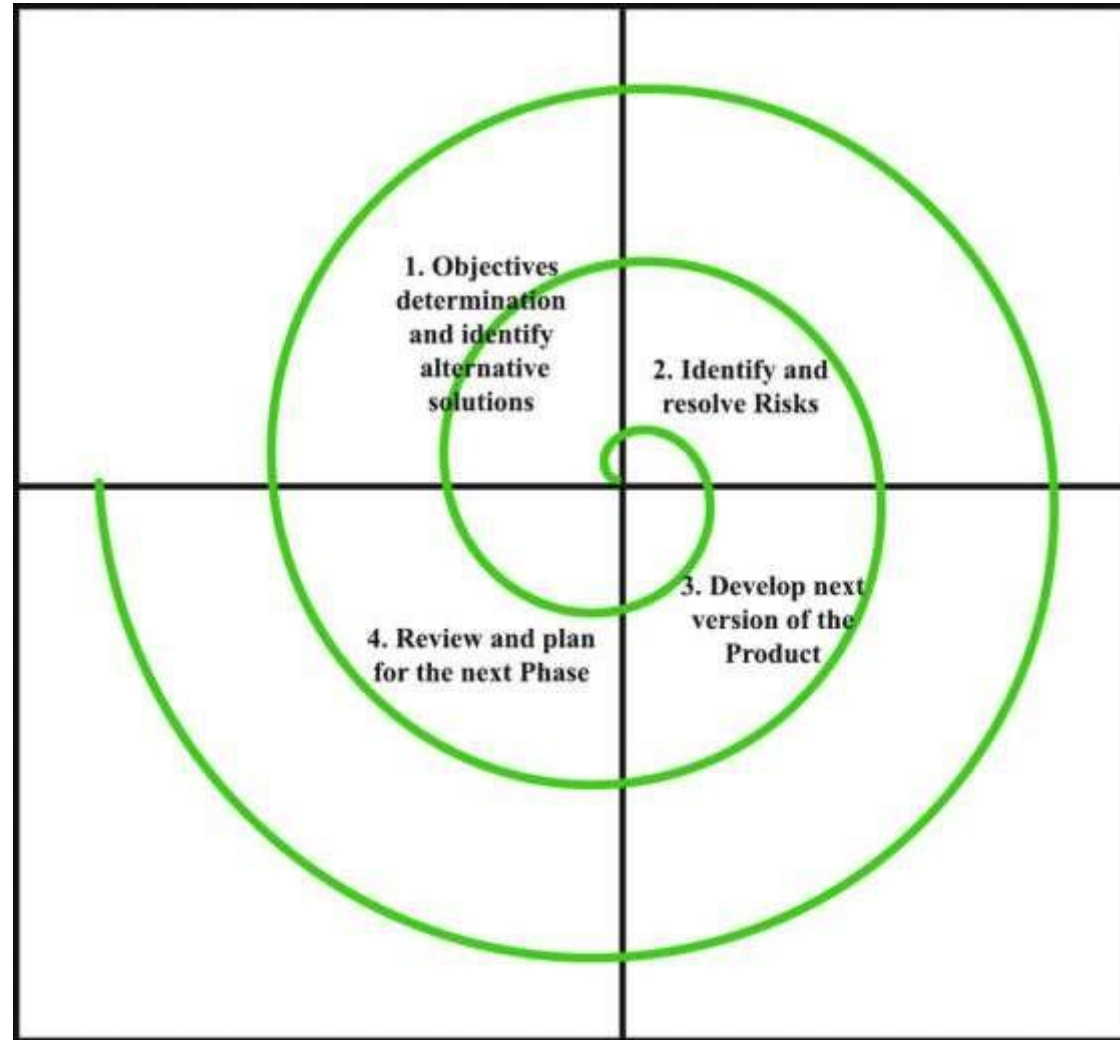
Disadvantages:

- User may not like fast activities.
- Not suitable for technical risks.

Spiral Model :

- This Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis.
- The spiral model has four phases: Planning, Design, Construct and Evaluation.

Quadrants in spiral model :



Advantages :

- Risk Identification at early stage.
- Suitable for high risk projects.
- Flexibility for adding functionality.

Disadvantages:

- Costly.
- Risk dependent.
- Not suitable for smaller projects.
- Difficult to meeting budget.

Win-Win Spiral Model:

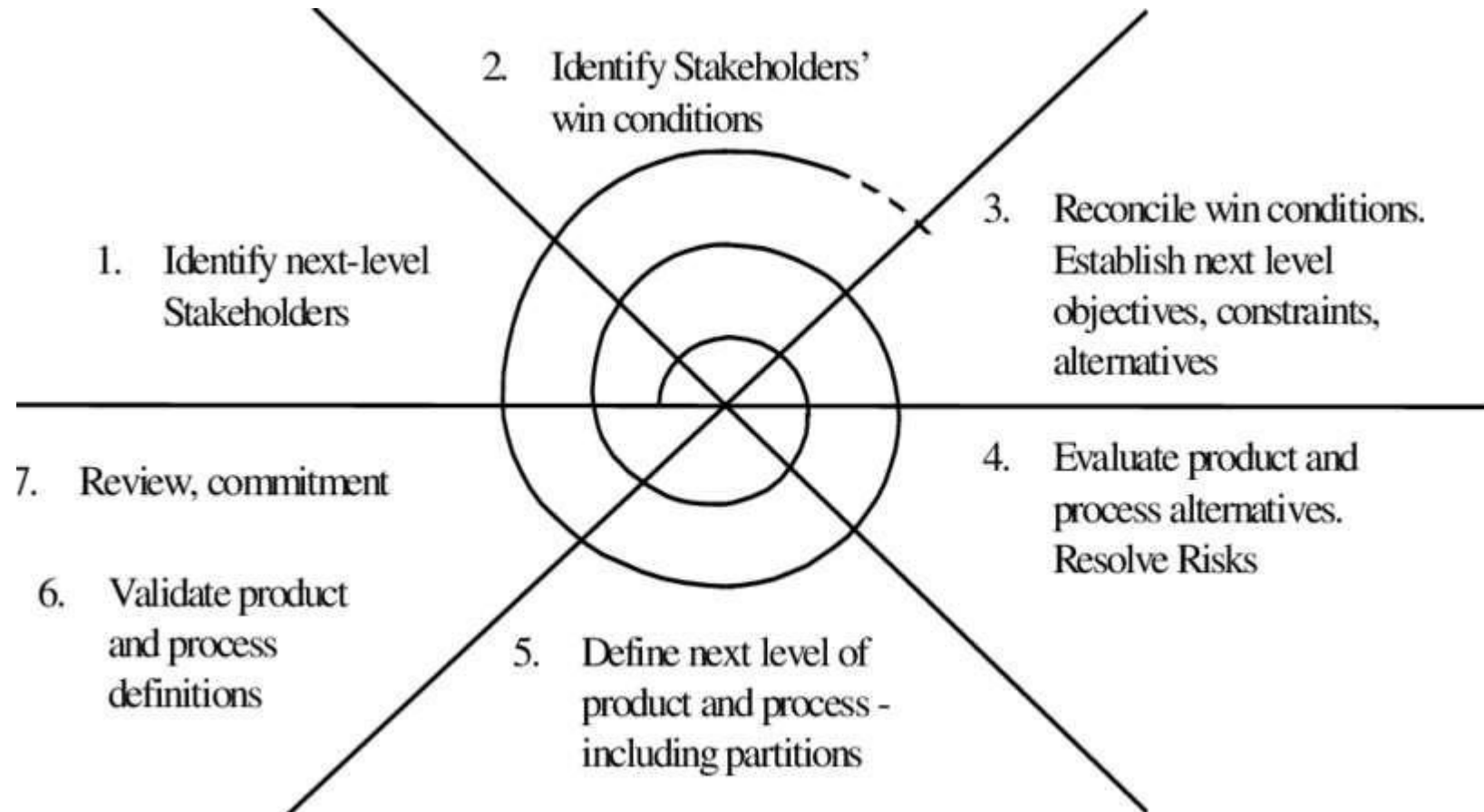
- The customer wins by getting the system satisfying most of their requirements and developers wins by working on achievable budgets and deadlines.

Advantages:

- Light weight methods suit small-medium size project.
- Produces good team.
- Test based approach to requirements and quality assurance

Diadvantages:

- Programming pairs is costly.
- Difficult to scale up to large projects where documentation.

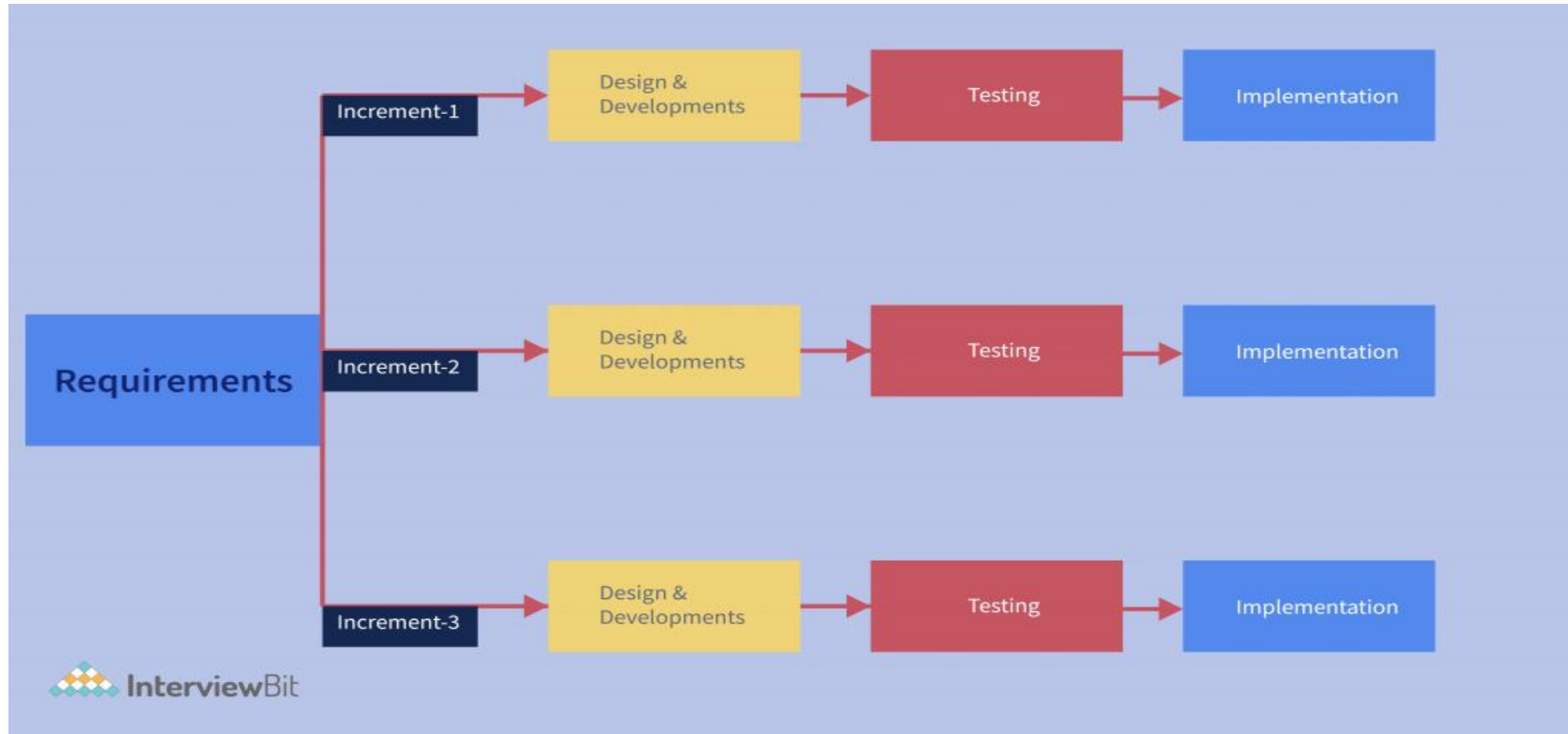


Incremental Model

Incremental Model, also known as the successive version model, is a widely adopted model of software development process where the software requirements are divided or broken down into multiple stand-alone modules/increments in the SDLC (Software Development Life Cycle).

Each increment is treated as a sub-project and goes through all phases of the SDLC incremental model.

Incremental Model



Introduction to DevOps

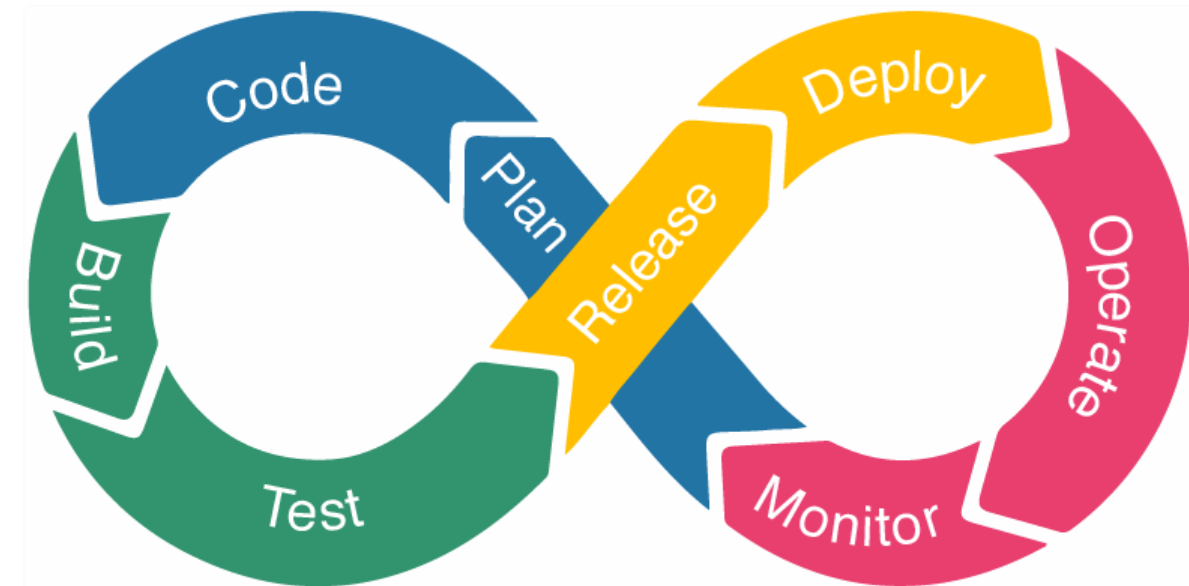
- DevOps is a **software development paradigm** that combines **Development (Dev)** and **Operations (Ops)**.
- It focuses on **continuous collaboration** between developers and IT operations teams.
- The goal is to **deliver software faster, reliably, and with high quality**.

Key Principles of DevOps

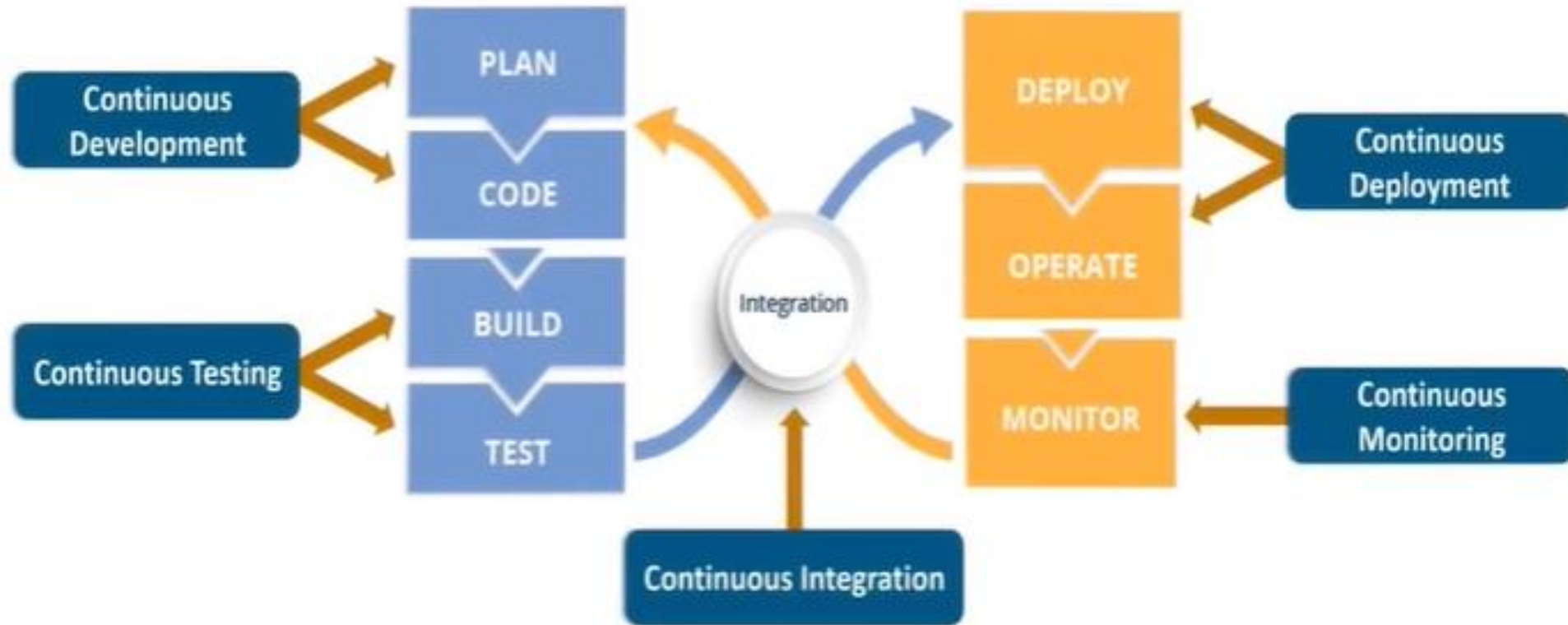
- Collaboration:** Developers and operations work as a single team.
- Automation:** Automates building, testing, deployment, and monitoring.
- Continuous Integration & Continuous Deployment (CI/CD):** Code changes are frequently integrated and released.
- Continuous Monitoring:** Applications are constantly monitored to detect issues early.

DevOps Lifecycle

- **Plan:** Define features and requirements.
- **Develop:** Write and review code.
- **Build & Test:** Automatically build and test code.
- **Deploy:** Release software to production automatically.
- **Operate:** Maintain and manage infrastructure.
- **Monitor:** Track performance and user feedback.



DevOps Lifecycle (Contd.)



DevOps Lifecycle

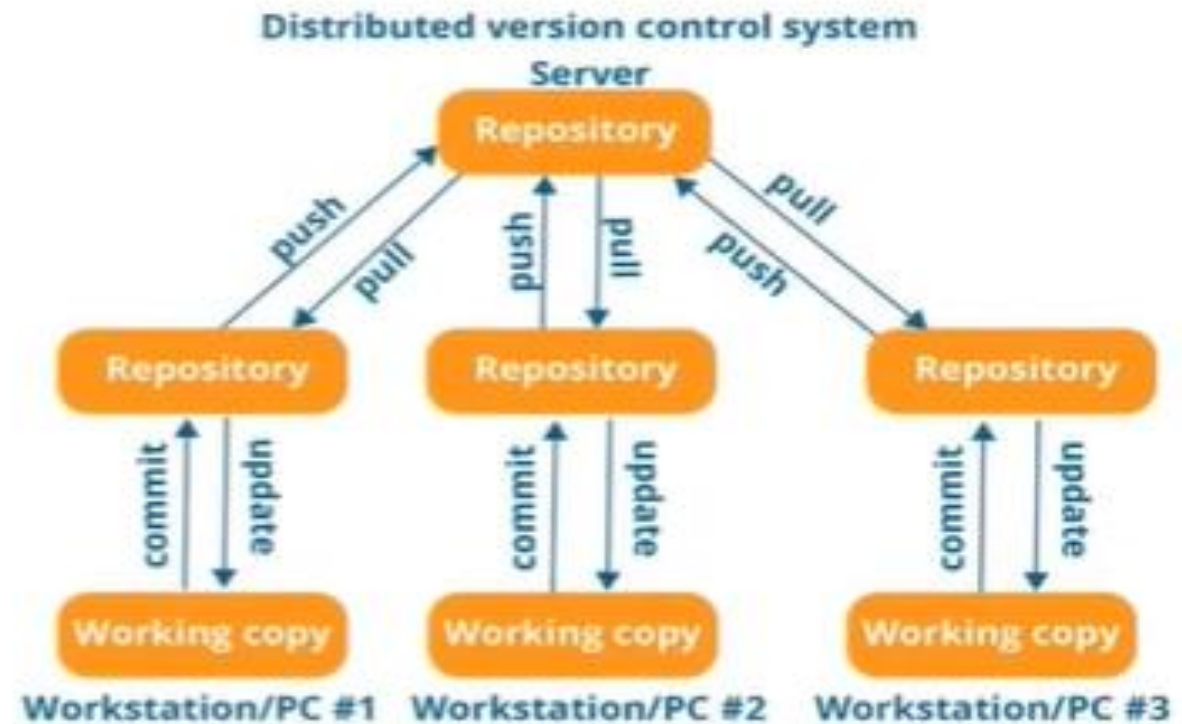
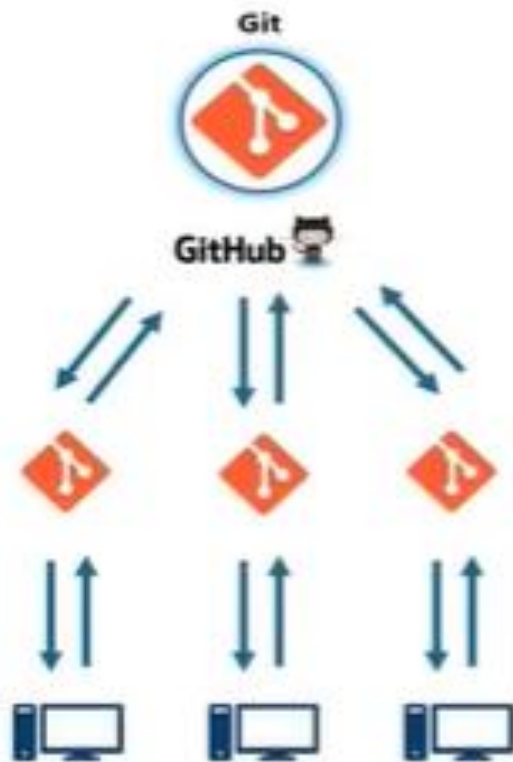
DevOps Lifecycle (Contd.)



DevOps Lifecycle

DevOps Lifecycle (Contd.)

Stage 1: Continuous Development



DevOps Lifecycle (Contd.)

Stage 1: Continuous Development (Contd.)

- This stage focuses on **planning and coding** the software application.
- Development is carried out in **small, continuous coding cycles**.
- Code can be written in **any programming language**.
- All code is maintained using **Version Control Systems (VCS)**.
- A **central repository** acts as a **single source of truth** for the project.
- Developers collaborate using the **latest committed code**.
- Operations teams can access the same code while **planning releases**.

DevOps Lifecycle (Contd.)

Stage 1: Continuous Development (Contd.)

- Version control enables:
 - Tracking code changes
 - Collaboration among team members
 - Rollback to previous stable versions in case of bugs or failures
- Commonly used tools:
 - Git
 - SVN
 - Mercurial
 - CVS
 - JIRA (for issue tracking and coordination)

DevOps Lifecycle (Contd.)

Stage 2: Continuous Integration

- Code from **multiple developers** is frequently merged into a **central repository**.
- New features are **integrated with existing code** early and often.
- Frequent integration helps **detect conflicts and defects early**.
- Continuous Integration reduces **integration problems and build costs**.
- Code is **automatically built, packaged, and tested** after integration.
- Integrated code is sent to **testing or production servers**.
- CI relies on **automated test execution** to reduce manual effort.
- Commonly used CI tools include:
 - **Jenkins**
 - **GitLab CI**

DevOps Lifecycle (Contd.)

Stage 3: Continuous Testing

- Code is **automatically tested** before deployment to detect bugs and performance issues.
- Testing is performed in **test environments**, often using **Docker containers**.
- **Automation testing** replaces repetitive manual testing and improves accuracy.
- Automated tests are triggered using **Continuous Integration tools**.
- Reduces **bugs** there by improving **reliability**
- **Test reports** are generated automatically, making it easier to identify failures.

Stage 4: Continuous Deployment

Tested code is automatically **deployed** to servers.

Deployment is handled using:

- Configuration Management tools**

- Containerization tools**

Ensures **consistent environments** across development, testing, and production.

Helps achieve **consistent and high-quality releases**.

Reduces manual deployment effort and minimizes human errors.

Containerization **packages applications with all dependencies**, ensuring uniform behavior across environments.

Docker is the most widely used containerization tool.

DevOps Lifecycle (Contd.)

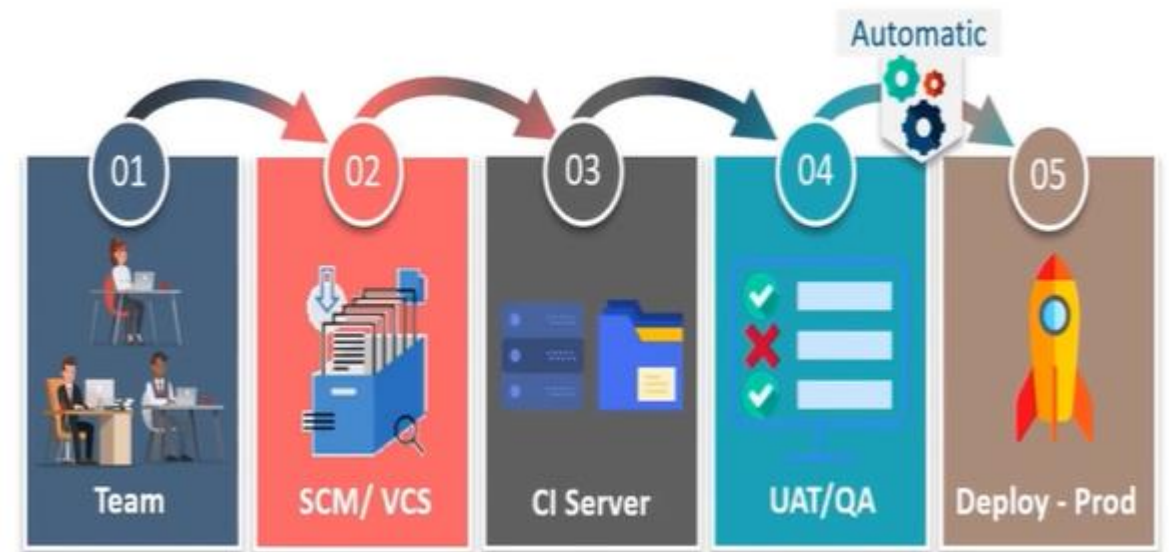
Stage 4: Continuous Deployment (Contd.)

Configuration Management Tools

- Puppet
- Chef
- Ansible
- SaltStack

Cloud Platforms Supporting Containers

- AWS ECS
- Azure Container Service
- Google Container Engine



DevOps Lifecycle (Contd.)

Stage 5: Continuous Feedback

- System collects **user feedback and performance data**
- Helps understand **real-world usage**
- Feedback is used to plan improvements

DevOps Lifecycle (Contd.)

Stage 6: Continuous Monitoring

- Application performance is **continuously monitored** after deployment.
- Helps detect:
 - Memory issues
 - Server failures
 - Crashes
 - Unusual system behavior
- Monitoring is critical because some bugs may **escape testing**.
- Monitoring tools provide **early warnings** before users face issues.
- Operations team plays a **major role** in this stage.

DevOps Lifecycle (Contd.)

Stage 6: Continuous Monitoring (Contd.)

- Common monitoring tools:

- Splunk**-Splunk **Splunk** is a **data monitoring and log analysis tool** used to collect, search, analyze, and visualize machine-generated data from servers, applications, and networks.

- Eg-A company collects logs from its servers. Splunk analyzes the logs and shows if any **errors, failures, or unusual activities** occur.

- Nagios**:To monitor server, network and application

- If server goes down, nagios sends notification or alert to admin

- New Relic**-Cloud based monitoring tool

DevOps Lifecycle (Contd.)

Stage 6: Continuous Monitoring tool (Contd.)

- **ELK Stack (Elasticsearch, Logstash, Kibana)** - - helps in **Log collection, Search and analysis ,Visualization and management**

a) Elasticsearch

- Stores and searches large volumes of data quickly.

b) Logstash

- Collects and processes logs from different sources.

c) Kibana

- Displays data through graphs, charts, and dashboards

DevOps Lifecycle (Contd.)

Stage 7: Continuous Operations

- Release and update processes are **fully automated**
- Keeps cycles short
- Allows developers to focus on **new features**

Benefits of DevOps

- **Faster and more efficient** software development
- **Fewer bugs** and improved user experience
- Better **collaboration** between Dev and Ops teams
- **Higher return on investment (ROI)**
- Automatic deployment supports **large-scale systems**
- Suitable for **both small and large teams**
- Early bug detection through **automated testing & monitoring**
- **Streamlined workflows** using version control and automation
- Continuous feedback improves **product quality**
- Automation reduces effort in testing and deployment

Tools Commonly Used in DevOps

- **Version Control:** Git, GitHub, GitLab
- **CI/CD Tools:** Jenkins, GitHub Actions, GitLab CI
- **Build Tools:** Maven, Gradle
- **Configuration Management:** Ansible, Chef, Puppet
- **Containerization:** Docker
- **Container Orchestration:** Kubernetes
- **Monitoring & Logging:** Prometheus, Grafana, ELK Stack
- **Cloud Platforms:** AWS, Azure, Google Cloud
- **Testing Tools:** Selenium, JUnit, TestNG

Understanding Software Architecture

- Software architecture refers to the high-level structure of a software system, including the components, interactions, and relationships between them.
- It provides a blueprint for the software system, which helps in understanding how the different parts of the system fit together and how they will work together to meet the requirements.

Why is Software Architecture Important?

- **Quality Attributes:** It defines the system's properties, such as scalability, reliability, and performance.
- **Risk Management:** It helps identify and mitigate potential risks in the early stages of software development.
- **Communication:** It provides communication between stakeholders, aiding understanding and collaboration.
- **Future-Proofing:** It makes future changes and enhancements more manageable by providing a clear structure and roadmap.

Key Concepts in Software Architecture

- **Components, Connectors, and Configurations:** Components are the main building blocks, connectors define how they communicate, and configuration describes how they are arranged in the system.
- **Architectural Styles and Patterns:** Standard design approaches that provide proven solutions for organizing system structure (e.g., layered, microservices).
- **Layers and Tiers in Architecture:** Layers separate responsibilities logically, while tiers separate system parts physically across different machines or environments.



Architectural Design Decisions

- How and when to make design decisions
 - Made after understanding business goals and system requirements.
 - Focus first on high-impact components; delay uncertain decisions when possible.
- Factors influencing these decisions
 - Business needs, budget, scalability, security, and performance requirements.
 - Team expertise, technology choices, time constraints, and compliance needs.
- Importance of trade-offs in architecture
 - Every architectural choice has advantages and disadvantages.
 - Architects must balance cost, performance, scalability, and maintainability.

Common Architectural Styles

- Monolithic Architecture
- Microservices Architecture
- Serverless Architecture
- Event-Driven Architecture

Common Architectural Styles (Contd.)

- **Monolithic Architecture:** The entire application is built as a single unified system where all components are tightly connected. It is simple to develop initially but can become difficult to scale and maintain as the system grows.
- **Microservices Architecture:** The application is divided into small, independent services that communicate through APIs. It improves scalability and flexibility but increases system complexity and management effort.

Common Architectural Styles (Contd.)

- **Serverless Architecture:** Applications run on cloud-managed services where the provider handles infrastructure and scaling. It reduces operational overhead but may lead to vendor dependency and limited control.
- **Event-Driven Architecture:** Components communicate by producing and responding to events. This allows high scalability and loose coupling but can make debugging and monitoring more complex.

Importance of Modularity

- **Definition of Modularity**

Modularity is designing software as independent, self-contained modules with specific responsibilities.

- **Benefits of Modularity in Software Design**

Improves maintainability, scalability, reusability, and easier testing.

- **Strategies to Achieve Modularity: Use**

- **Separation of concerns:** Divide the system so each part handles one specific task only.
- **Clear interfaces:** Define simple and clear rules for how different parts communicate with each other.
- **Loose coupling:** Keep modules independent so changes in one module don't heavily affect others.
- **High cohesion:** Make sure each module focuses on closely related tasks and has a clear purpose.

Software architecture characteristics

- **Aligning Architecture with Business Goals:** The system design should support business objectives and deliver value.
- **Scalability:** Ability of the system to handle growth in users, data, or workload.
- **Maintainability:** Ease of fixing bugs, updating features, and improving the system.
- **Extensibility:** Ability to add new features without major changes to existing code.
- **Flexibility:** Ability to adapt to changing requirements or technologies.
- **Performance Efficiency:** System should respond quickly and use resources effectively.
- **Security:** Protects data and system from unauthorized access and threats.
- **Usability:** System should be easy and user-friendly to operate.
- **Reusability:** Components can be reused in other parts of the system or in different projects.

Understanding Software Design

- Software design refers to the process of creating a plan, blueprint, or architecture for a software system.
- It involves making decisions about the structure, functionality, and behavior of the system, as well as the algorithms and data structures that will be used to implement it.

Understanding Software Design

- Software design is a critical step in the software development process, as it helps ensure that the system is developed efficiently, effectively, and meets the requirements of its users.
- It involves a series of activities, including:
 - Defining the system's requirements and constraints
 - Creating models, prototypes, and diagrams to represent the system's architecture and functionality
 - Identifying and evaluating design options and trade-offs
 - Selecting the most appropriate design solution
 - Documenting the design in sufficient detail to allow for its implementation

Design Principles

- Divide the system so each part handles a specific responsibility.
- Build the system using independent, self-contained components.
- Hide complex details and show only necessary features.
- Follow five object-oriented principles that improve design and maintainability.
- Avoid duplicating code or logic.
- Keep the design simple and avoid unnecessary complexity.
- Do not add features until they are actually required.
- Write tests before writing the actual code to ensure correctness.

Design Patterns

- Design patterns at the software design level are reusable solutions to common problems that arise during the software design process.
- They provide a proven, standardized approach to solving specific design problems, making it easier for developers to create robust, maintainable, and extensible software systems.

Desirable Characteristics of a Design

- Minimal complexity
 - A design should be simple and easy to understand, avoiding unnecessary complexity that can make it hard to maintain and extend.
- Ease of maintenance
 - A design should be easy to maintain and modify, with a focus on making it self-explanatory for maintenance programmers.
- Loose coupling
 - A design should minimize connections between different parts of a program to reduce the impact of changes and improvements.

Desirable Characteristics of a Design (Contd.)

- Extensibility
 - A design should be modular and allow for easy extension and modification without affecting the underlying structure.
- Reusability
 - A design should promote reusability by utilizing modular, flexible, and standardized components.
- High fan-in
 - A design should have a high number of classes that use a given class, indicating good use of utility classes.

Desirable Characteristics of a Design (Contd.)

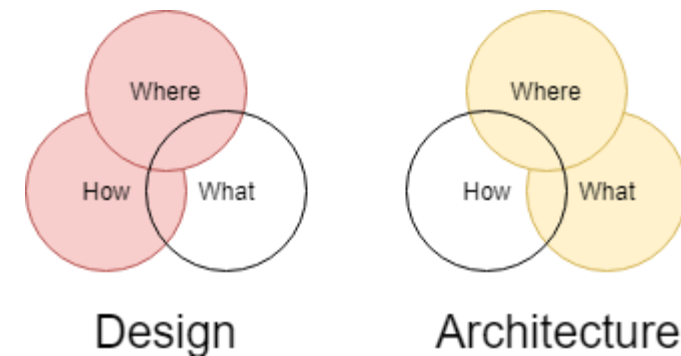
- Low-to-medium fan-out
 - A design should have a low-to-medium number of classes used by a given class, avoiding high fan-out that may indicate complexity.
- Portability
 - A design should be easy to move to another environment, minimizing environment-specific dependencies.
- Leanness
 - A design should avoid unnecessary parts and code, minimizing the effort required for development, review, testing, and maintenance.

Desirable Characteristics of a Design (Contd.)

- Stratification
 - A design should have clear levels of decomposition, allowing for a consistent view at each level without dipping into other levels.
- Standard techniques
 - A design should utilize established, widely-used techniques and tools to create a familiar and easy-to-understand design.

Difference between Architecture and Design

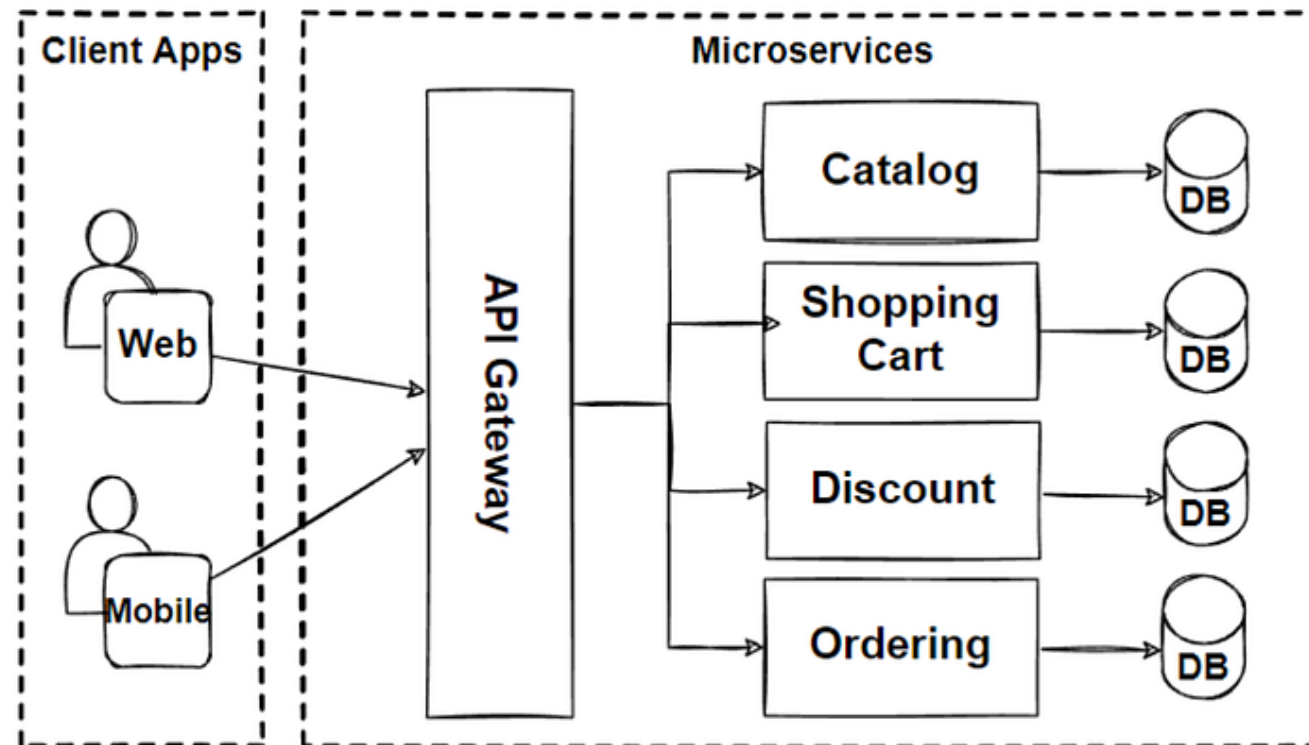
- Software design typically involves making decisions about the algorithms, data structures, and other low-level details of the system, while software architecture focuses on the overall shape of the system and how it will meet the requirements.



Microservices and Serverless Architectures

Microservices Architecture (MSA)

Definition: Microservices Architecture is an architectural style in which an application is developed as a collection of **small, autonomous services**, each running in its **own process** and communicating through **lightweight mechanisms** such as **REST** or **gRPC**.



Microservices Architecture

Microservices Architecture (MSA)



Key Characteristics:

- **Autonomous Services**

Each service is **independently deployable**, allowing faster releases and minimal impact on other services.

- **Loose Coupling**

Services are loosely coupled, enabling independent development, testing, and maintenance.

- **Business Capability–Driven**

Services are organized around **specific business functions**, often aligned with **Domain-Driven Design (DDD)** and **bounded contexts**.

Microservices Architecture (MSA)



Key Characteristics: (Contd.)

Decentralized Data Management

Each service follows the **database-per-service** pattern, ensuring data ownership and independence.

- **Resilience and Fault Isolation**

Failures in one service do not propagate to others, supported by patterns such as the **Bulkhead pattern**.

- **Small, Cross-Functional Teams**

Each service is typically owned by a **small, dedicated team**, improving accountability and agility.

- **Ease of Maintenance and Testing**

Smaller codebases make services **easier to maintain, test, and evolve**.

Microservice vs API – What's the Difference?

An API is NOT a Microservice

What is an API?

- An **API (Application Programming Interface)** exposes **data and behavior** to consumers.
- Acts as a **contract** between a system and its clients.
- Defines **how** consumers interact with a system.
- An API does **not define architecture**.

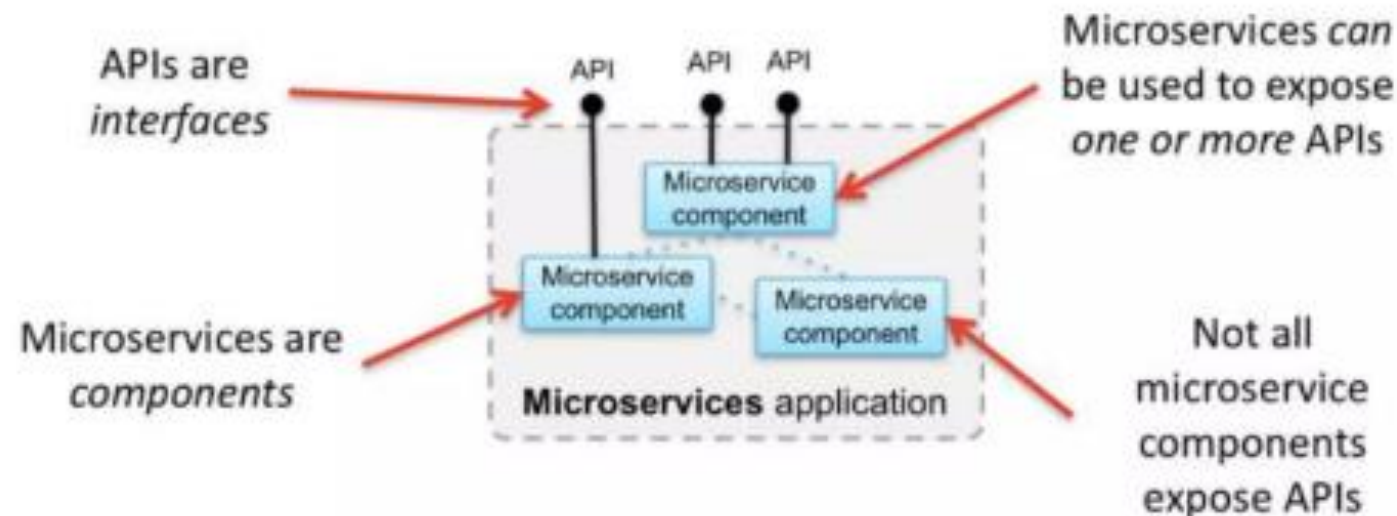
What is a Microservice?

- A **microservice** is an **architectural style**.
- Represents a **fine-grained, independently deployable service**.
- Built around a **specific business capability**.
- Owns its **logic, data, and lifecycle**.

Relationship Between API and Microservices

Microservices **may expose APIs**, but:

- There is **no one-to-one relationship**.
- **Multiple microservices** can together form a **single API**.
- **One API endpoint** may aggregate responses from **multiple microservices**.



When to Use Microservices?

Use Microservices When:

Monolith becomes too complex to manage

The application grows so large that it becomes **hard to modify and deploy**.

Large teams are involved

Multiple teams need to work **independently** on different business functions.

Independent evolution of business functions is required

Different modules must change and scale **without affecting the whole system**.

Scalability requirements increase

Different parts of the system need **independent scaling**.

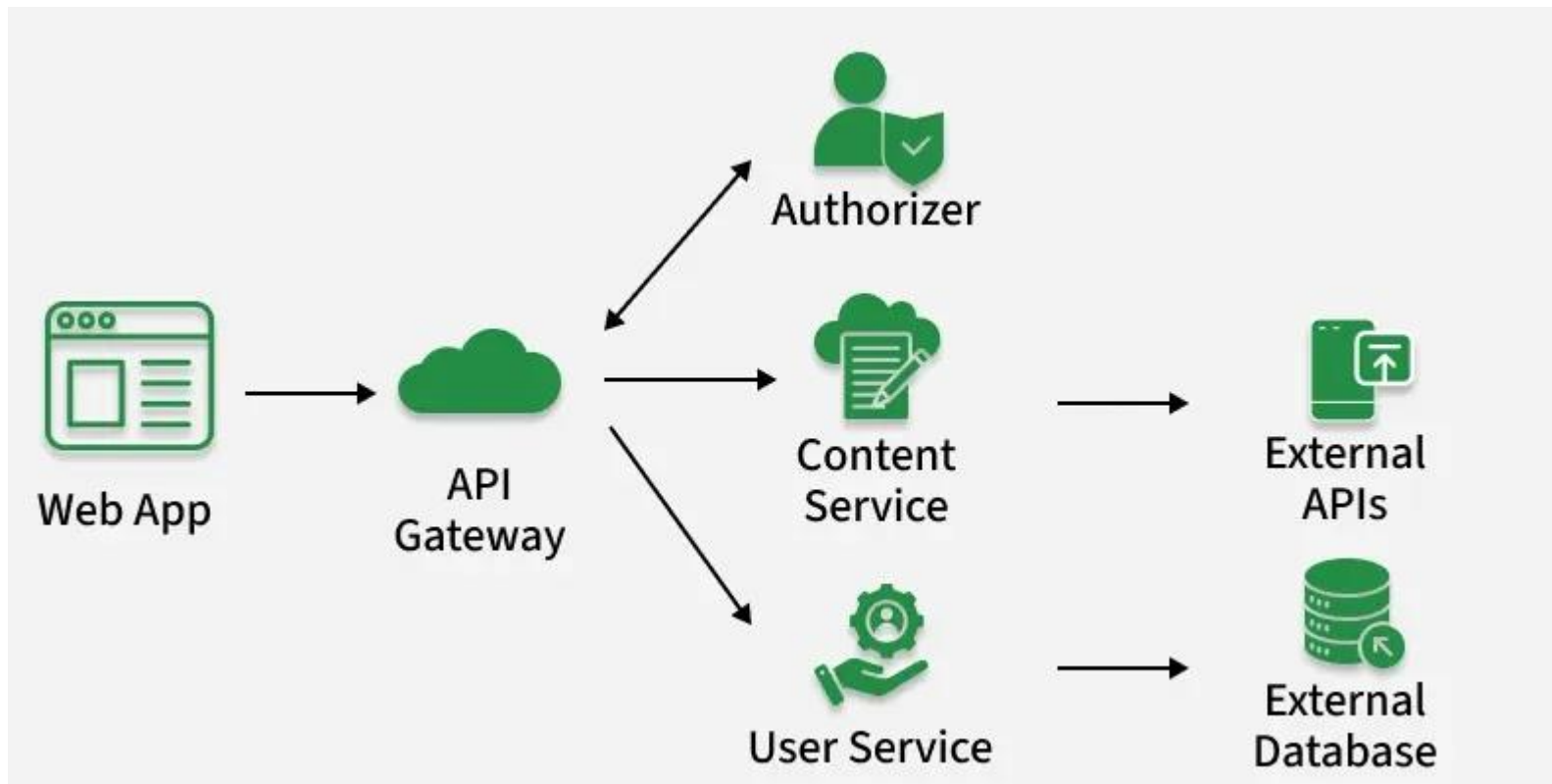
Multiple user interaction models or tenants exist

The system must support **diverse clients or multi-tenant environments**.

Serverless Architecture

- A **cloud computing model** where the **cloud provider** manages **server provisioning, scaling, and maintenance**.
- Developers focus entirely on **writing and deploying code**.
- Code runs in **fully managed, stateless environments** and is **triggered by events**.
- Cloud providers dynamically handle **infrastructure, scaling, and resource allocation**.
- Code executes in **event-driven, stateless compute containers**.
- Developers are **abstracted from server management tasks** like capacity planning and maintenance.

Serverless Architecture



Serverless Computing Providers and Platforms

•AWS Lambda

- Executes code without provisioning or managing servers
- Triggered by events from 200+ AWS services and SaaS apps

•Azure Functions

- Event-triggered serverless compute
- Integrates with Azure services
- Supports multiple programming languages

•Google Cloud Functions

- Serverless execution environment
- Google manages scaling and infrastructure

•Oracle Cloud Functions

- Functions triggered by HTTP or Oracle Cloud events
- Integrates with Oracle Cloud services

Step 1: Understand the Serverless Model

- Suitable for **unpredictable traffic**
- Well-suited for **microservices-based applications**

Step 2: Choose the Right Provider

- Evaluate supported **languages, integrations, provider features**
- Compare **cost, performance, cold start behavior**
- Providers: AWS Lambda, Azure Functions, Google Cloud Functions, Oracle Cloud Functions

Step 3: Designing Your Application

- **Event-driven design**
- **Stateless functions** (use databases/cloud storage for state)
- **Microservices-oriented design**

Steps for Developing Serverless Applications (Contd.)

Step 4: Development Environment Setup

Use tools like:

- Serverless Framework

- AWS SAM

- Azure Functions Core Tools

Configure **local testing and simulation tools**

Step 5: Implementing Functions

Write **small, single-purpose functions**

Integrate **managed services** for DB, authentication, storage

Step 6: Managing Dependencies

Include only **necessary libraries**

Smaller packages improve **cold start performance**

Step 7: Deployment and Continuous Integration

Automate using **CI/CD pipelines**

Tools: GitHub Actions, Jenkins, CircleCI

Serverless Application Design Patterns

•Function-as-a-Gateway (FaG)

- Routes incoming HTTP requests
- Used for lightweight API gateways

•Event Stream Processing

- Processes streams of logs, transactions, IoT data
- Used for real-time analytics and monitoring

•Aggregator

- Combines responses from multiple services/functions
- Returns a unified response

•Strangler Fig Pattern

- Gradual migration from legacy systems
- New features built using serverless

•Circuit Breaker

- Prevents cascading failures
- Stops repeated failed function calls

Scaling and Performance Considerations

Scaling in Serverless

- **Automatic Scaling**

Cloud platform automatically increases or decreases resources based on number of requests/events.

No manual scaling required.

- **Cold Start Latency**

When a function runs after being idle, it may take extra time to start.

This causes a small delay in the first request.

- **Throttling**

Cloud providers limit how many function executions can run at a time.

If requests exceed the limit, some requests may be delayed.

Performance Optimization Strategies

- Optimize Function Code**

Keep functions small and efficient to reduce execution time.

- Manage Dependencies**

Avoid unnecessary libraries.

Smaller code loads faster and reduces startup time.

- Persistent Connections**

Reuse database or service connections to reduce repeated connection delays.

Security Best Practices in Serverless architecture.

- **Least Privilege Principle (PoLP)**

Give each function only the permissions it really needs.

- **Secure Application Secrets**

Do not hard-code API keys or passwords.

Use cloud secret managers.

- **Input Validation**

Validate inputs to prevent attacks like SQL injection and XSS.

- **Secure API Gateway**

Use authentication, rate limiting, and access control for APIs.

- **Encryption**

Encrypt data while sending (HTTPS) and while storing.

Serverless Architecture – Use Cases

- Chatbots and Virtual Assistants
- Real-time Data Processing and Analytics
- Web and Mobile Backends
- Scheduled Tasks / Cron Jobs

Challenges of Serverless Architecture

- Cold Start Delay**

First request may be slow after inactivity.

- Limited Execution Environment**

Limits on memory, execution time, and language support.

- Debugging and Monitoring Difficulty**

Harder to trace errors across many small functions.

- State Management Complexity**

Functions are stateless; state must be stored externally.

- Security and Compliance Challenges**

More effort needed to manage security and regulatory requirements.

END OF UNIT-3